# I OBJETIVOS

En esta práctica el alumno:

- Utilizará el sistema de paso de mensajes Posix como una herramienta para la sincronización y comunicación de procesos concurrentes.

# II BIBLIOGRAFÍA


- William Stallings, “SISTEMAS OPERATIVOS”,Prentice Hall.
- Silberschatz, Galvin, Gagne, “SISTEMAS OPERATIVOS”, Mc Graw Hill.
- Neil Matthew & Richard Stones, “BEGINNING LINUX PROGRAMMING”, Wrox.


# III RECURSOS

- Una estación de trabajo con Linux con su ambiente gráfico.
- Un editor de texto.
- El compilador de C GNU.

# IV ACTIVIDADES

## *1 Serie de Mercator*

En matemáticas, la serie de Mercator o serie de Newton–Mercator es la serie de Taylor del logaritmo natural:

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots.$$

Escrita utilizando notación sumatoria:

$$\ln(1+x) = \sum_{n=1}^{\infty} \frac{(-1)^{n+1}}{n} x^n.$$

La serie converge al logaritmo natural (desplazado en 1) cuando $-1 < x \leq 1$.

## 2 Problema a resolver

El programa que se muestra a continuación realiza el cálculo de $ln(1+x)$ donde $x = 1$ por medio de la serie de Mercator. Para hacer este cálculo de manera precisa, la sumatoria se realiza con 200,000 términos, lo cual implica mucho trabajo de procesamiento. Para que en este trabajo se aproveche la capacidad que brinda un procesador Multicore, éste se reparte en 4 procesos que pueden ejecutarse en paralelo.

El programa que se muestra en la Figura 1 crea un proceso que llamaremos proceso maestro, y cuatro procesos que serán los procesos esclavos quienes realizan los cálculos. El proceso maestro lee de un archivo de texto el valor de x que se utilizará para hacer el cálculo y establece el valor de una bandera que les indica a los procesos esclavos que pueden iniciar los cálculos, y mientras los esclavos hacen el trabajo el proceso maestro esperará a que estos terminen. Todos los procesos se comunican y sincronizan a través de variables en memoria compartida.

Aunque esta solución funciona, esta no es eficiente ya que tanto el maestro como esclavos tienen espera ocupada. Además, es probable que en algunas ejecuciones exista un error cuando se realiza el conteo de procesos.

Ahora hay que utilizar la librería de **paso de mensajes Posix** para arreglar el código de manera que no genere espera ocupada. La intención es lograr tanto la sincronización como la comunicación con mensajes por lo en esta solución ya no podremos **utilizar memoria compartida, por lo tanto la que se usa en el ejemplo hay que eliminarla.**

```
/*
        Para compilar incluir la librería  m (matemáticas)

        Ejemplo:
            gcc -o mercator mercator.c  -lm
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <math.h>
#include <sys/time.h>
#include <sys/wait.h>
#include <sys/shm.h>

#define NPROCS 4
#define SERIES_MEMBER_COUNT 200000
```

```
typedef struct {
    double sums[NPROCS];
    int proc_count;
    int start_all;
    double x_val;
    double res;
} SHARED;

SHARED *shared;

double get_member(int n, double x)
{
    int i;
    double numerator = 1;

    for(i=0; i<n; i++ )
        numerator = numerator*x;

    if (n % 2 == 0)
        return ( - numerator / n );
    else
        return numerator/n;
}

void proc(int proc_num)
{
    int i;

    // Espera a que master_proc ponga la bandera en 1 para iniciar
    while(!(shared->start_all));

    // Cada proceso realiza el cálculo de los términos que le tocan
    shared->sums[proc_num] = 0;
    for(i=proc_num; i<SERIES_MEMBER_COUNT;i+=NPROCS)
        shared->sums[proc_num] += get_member(i+1, shared->x_val);

    // Incrementa la variable proc_count que es la cantidad de procesos que
terminaron su cálculo
    shared->proc_count++;

    exit(0);
}
```

```
void master_proc()
{
    int i;

    // Obtener el valor de x desde el archivo entrada.txt
    FILE *fp = fopen("entrada.txt","r");
    if(fp==NULL)
        exit(1);

    fscanf(fp,"%lf",&shared->x_val);
    fclose(fp);

    // el master_proc pone una bandera para que los procesos inicien con el
cálculo
    shared->start_all=1;

    // Espera a que todos los procesos terminen su cálculo
    while (shared->proc_count != NPROCS) {}

    // Una vez que todos terminan, suma el total de cada uno
    shared->res = 0;
    for(i=0; i<NPROCS; i++)
        shared->res += shared->sums[i];

    exit(0);
}

int main()
{
    int *threadIdPtr;

    long long start_ts;
    long long stop_ts;
    long long elapsed_time;
    long lElapsedTime;
    struct timeval ts;
    int i;
    int p;
    int shmid;
    int status;
```

```
    // Solicita y conecta la memoria compartida
    shmid = shmget(0x1234,sizeof(SHARED),0666|IPC_CREAT);
    shared = shmat(shmid,NULL,0);

    // inicializa las variables en memoria compartida
    shared->proc_count = 0;
    shared->start_all = 0;

    gettimeofday(&ts, NULL);
    start_ts = ts.tv_sec; // Tiempo inicial

    for(i=0; i<NPROCS;i++)
    {
        p = fork();
        if(p==0)
            proc(i);
    }

    p = fork();
    if(p==0)
        master_proc();

    printf("El recuento de ln(1 + x) miembros de la serie de Mercator es
%d\n",SERIES_MEMBER_COUNT);

    for(int i=0;i<NPROCS+1;i++)
    {
        wait(&status);
        if(status==0x100)   // Si el master_proc termina con error
        {
            fprintf(stderr,"Proceso no puede abrir el archivo de entrada\n");
            break;
        }
    }

    gettimeofday(&ts, NULL);
    stop_ts = ts.tv_sec; // Tiempo final
    elapsed_time = stop_ts - start_ts;

    printf("Tiempo = %lld segundos\n", elapsed_time);
    printf("El resultado es %10.8f\n", shared->res);
```

```
    printf("Llamando a la función ln(1 + %f) = %10.8f\n",shared->x_val,
log(1+shared->x_val));

    // Desconecta y elimnina la memoria compartida
    shmdt(shared);
    shmctl(shmid,IPC_RMID,NULL);
}
```

**Figura 1**

**Importante:** Esta solución se tiene que hacer con procesos y emplear los mecanismos de comunicación y sincronización de procesos que aquí mismo se indican, no se aceptarán soluciones que utilicen hilos. También es importante que tanto la comunicación y sincronización sea a través del sistema de paso de mensajes por lo que es necesario **eliminar la memoria compartida y no emplear semáforos**

# V ENTREGA Y EVALUACIÓN



**No incluya líneas de código en sus programas de las cuales desconozca su funcionamiento. El código no conocido será anulado en el funcionamiento de la práctica.**



**Aunque en semestres anteriores se han realizado prácticas similares a esta, hay aspectos que hacen que esta sea diferente. Cualquier evidencia que muestre el intento de entregar una práctica de semestre anterior será calificada como plagio.**

## *1 Entrega y Revisión*

Entregar en el apartado correspondiente de Canvas los programas fuentes con su respectivo `Makefile` en un archivo ZIP.

## *2 Equipos*

Esta práctica se hará en equipos (máximo 2 integrantes), es necesario que en la revisión esté el equipo completo ya que el integrante que no se presente no tendrá calificación en la práctica.

Importante: Al indicarse que el trabajo debe ser desarrollado por equipos, se entiende que no se permite colaboración entre equipos, cualquier evidencia de esto será considerada plagio.

# 3 Evaluación

| | | | | | |
|---|---|---|---|---|---|
| Puntualidad en las revisiones | El equipo estuvo completo y puntual en todas las sesiones de revisión. | | Si hubo dos o más sesiones con el equipo, el equipo estuvo completo y puntual en casi todas las sesiones de revisión | | Si solo hubo una sesión de revisión, el equipo no estuvo completo o no fue puntual. Si fueron dos o más sesiones de revisión, en más de una sesión el equipo no estuvo completo o fue puntual |
| | +5 | | +2.5 | | 0 |
| Especificaciones de entrega | La entrega del producto cumple con todas las especificaciones indicadas en el documento de la práctica, por ejemplo, los archivos se entregan de acuerdo a las formas indicadas en el documento de la práctica. | | | | La entrega del producto no cumple con al menos una de las especificaciones indicadas en el documento de la práctica |
| | +5 | | | | 0 |
| Funcionamiento | El producto cumple con todas las especificaciones indicadas en el documento y no tiene fallas | El producto muestra una falla no esperada. | Se da una de las siguientes condiciones:<br><br>-Está incompleto (falta máximo aprox 50%), pero lo demás puede funcionar bien.<br><br>-Está completo pero muestra dos fallas | Se da una de las siguientes condiciones:<br><br>-Está completo pero muestra 3 fallas o más.<br><br>-Está incompleto (falta máximo aprox 50%) y además muestra fallas<br><br>-Está incompleto (falta máximo aprox 66%) | El producto no funciona o está incompleto (más del 66%). |
| | +80 | +60 | +40 | +20 | 0 |
| Interfaz con el usuario | El producto funciona y pudo ser utilizado sin necesidad de recibir indicaciones por el desarrollador, tiene instrucciones claras para ser utilizado. | | El producto funciona, pero hubo necesidad de recibir alguna indicación para su uso por parte del desarrollador del producto | | El producto carece de instrucciones claras para ser utilizado y requiere que alguno de los desarrolladores esté presente para su utilización o no puede utilizarse debido a que no está completo |
| | +5 | | +3.5 | | 0 |
| Claridad en el código | El código es claro, usa nombres de variables adecuadas, está debidamente comentado e indentado. Puede ser entendido por cualquier otra persona que no intervino en su desarrollo. | | El código carece de claridad, puede ser entendido por cualquier persona ajena a su desarrollo pero con cierta dificultad. | | El código carece de comentarios, está mal indentado, usa nombres de variables no adecuadas. |
| | +5 | | +3.5 | | 0 |
| Defensa del producto | Todos los que presentan la práctica son capaces de explicar cualquier parte del producto presentado | | Uno de los que presenta la práctica muestra dudas sobre alguna parte del desarrollo del producto presentado | | Más de un integrante no muestra evidencia de que conoce el producto, o si el trabajo fue individual, el desarrollador duda sobre el desarrollo del producto que presenta. |
| | x 1(puntos se multiplican por 1) | | x 0.5 (puntos se multiplican por 0.5) | | x 0 (puntos se multiplican por 0) |